MATH 4610: Fundamentals of Numerical Analysis Fall 2026

Final Project C

Due: Friday, December 18, 2026, 11:59 pm, on Canvas

General Instructions

This is one of three final projects. Choose one of them. Submit on Canvas (i) a report in PDF, typeset in LaTeX or Word, and (ii) a zip file with your code. All code must be written in MATLAB. Include one script for each problem (`Q1.m`, `Q2.m`, `Q3.m`) that produces the figures and tables of that problem, and a short `README` file. For each problem, the report should contain the derivations, the tables and figures, and a brief discussion of the results.

Write the networks, the gradients, and the training methods yourself. Do not use the Deep Learning Toolbox, the Optimization Toolbox, or automatic differentiation. MATLAB's linear algebra functions (`\`, `qr`, `chol`, `eig`, `cond`) may be used. Set the random seed with `rng` at the start of each script and report it.

You may discuss the project with others, but your code and your report must be your own. Cite any sources you use.

Final presentation. Each student will present the project in class during the week of December 7–11, on Monday (Dec. 7), Wednesday (Dec. 9), or Friday (Dec. 11). The schedule will be posted on Canvas. Present the problem, your main results, and your conclusions; the work need not be finished. The written report is due on December 18 and may take into account comments made at the presentation.

Grading

Problem 1: ReLU networks and piecewise linear functions	20
Problem 2: computing the gradient	25
Problem 3: training	25
Code (correct, readable, reproducible)	10
Written report	10
Final presentation	10
Total	100
Bonus: the Levenberg–Marquardt method	10

Setting

All problems use the function

$$f(x) = \frac{1}{1 + 25x^2}, \qquad -1 \le x \le 1,$$

and the data (x_i, y_i), $y_i = f(x_i)$, at the $m = 200$ equally spaced points $x_i = -1 + 2(i-1)/(m-1)$, $i = 1, \dots, m$. A neural network with one hidden layer of n neurons and activation function σ is the

function

$$N(x;\theta) = c + \sum_{k=1}^{n} a_k\, \sigma(w_k x + b_k),$$

with parameters $\theta = (a, w, b, c)$, where $a, w, b \in \mathbb{R}^n$ and $c \in \mathbb{R}$. Training the network means minimizing

$$L(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \big(N(x_i;\theta) - y_i\big)^2.$$

The error of a network is measured by $E_\infty = \max_j |f(z_j) - N(z_j;\theta)|$, where z_j are 2001 equally spaced points in $[-1, 1]$.

Problem 1: ReLU Networks and Piecewise Linear Functions (20 points)

In this problem $\sigma(s) = \mathrm{ReLU}(s) = \max(0, s)$.

(a) Let $-1 = x_0 < x_1 < \cdots < x_n = 1$, let s be the piecewise linear interpolant of f at these nodes, and let $m_k = \big(f(x_{k+1}) - f(x_k)\big)/(x_{k+1} - x_k)$. Prove that for $-1 \le x \le 1$

$$s(x) = f(x_0) + m_0\, \mathrm{ReLU}(x - x_0) + \sum_{k=1}^{n-1} (m_k - m_{k-1})\, \mathrm{ReLU}(x - x_k).$$

Thus s is a ReLU network with n neurons and $w_k = 1$. Show, conversely, that every ReLU network of this form is a continuous piecewise linear function, and find its break points.

(b) Take equally spaced nodes. Compute E_∞ of s for $n = 8, 16, \ldots, 512$, make a table of the errors and observed orders, and compare with the bound $\|f - s\|_\infty \le \frac{h^2}{8} \|f''\|_\infty$.

(c) Now fix $w_k = 1$ and $b_k = -\beta_k$, and determine only a and c. Show that this is the linear least squares problem

$$\min_{\mathbf{u}} \ \frac{1}{2m} \|\Phi \mathbf{u} - \mathbf{y}\|_2^2, \qquad \Phi_{ik} = \mathrm{ReLU}(x_i - \beta_k),$$

where Φ has an additional column of ones and $\mathbf{u} = (a, c)$. For $n = 8, 16, 32, 64, 128$, solve it with `qr` for (i) equally spaced β_k in $[-1, 1)$ and (ii) β_k chosen at random, uniformly in $[-1, 1]$; in case (ii) average E_∞ over 10 random choices. Plot E_∞ against n on a log-log scale for both cases and for the interpolant of (b).

(d) Plot $\kappa_2(\Phi)$ against n for both choices of β_k. Solve the problem also by the normal equations $\Phi^T \Phi\, \mathbf{u} = \Phi^T \mathbf{y}$, using `chol`, and compare with the solution by `qr`. Explain the difference using $\kappa_2(\Phi^T \Phi) = \kappa_2(\Phi)^2$, and explain why break points that are close together make Φ ill-conditioned.

Problem 2: Computing the Gradient (25 points)

From now on $\sigma = \tanh$, and all $3n + 1$ parameters are trained.

(a) Let $z_{ik} = w_k x_i + b_k$, $h_{ik} = \tanh(z_{ik})$, and $r_i = N(x_i;\theta) - y_i$. Use the chain rule to show that

$$\frac{\partial L}{\partial a_k} = \frac{1}{m}\sum_{i=1}^{m} r_i\, h_{ik}, \qquad \frac{\partial L}{\partial b_k} = \frac{1}{m}\sum_{i=1}^{m} r_i\, a_k(1-h_{ik}^2),$$
$$\frac{\partial L}{\partial w_k} = \frac{1}{m}\sum_{i=1}^{m} r_i\, a_k(1-h_{ik}^2)\, x_i, \quad \frac{\partial L}{\partial c} = \frac{1}{m}\sum_{i=1}^{m} r_i.$$

Write $N(x_i;\theta)$ and these formulas with matrices and vectors, without loops over i or k. Count the number of floating point operations needed for one evaluation of ∇L, in terms of m and n.

(b) Write MATLAB functions that compute $L(\theta)$ and $\nabla L(\theta)$ with the formulas of (a).

(c) Take $n = 10$, a random θ, and a random unit vector $\mathbf{d}$. Approximate $\nabla L(\theta)^T\mathbf{d}$ by

$$D_+(\delta) = \frac{L(\theta+\delta\mathbf{d}) - L(\theta)}{\delta}, \qquad D_0(\delta) = \frac{L(\theta+\delta\mathbf{d}) - L(\theta-\delta\mathbf{d})}{2\delta},$$

for $\delta = 10^{-1}, 10^{-2}, \ldots, 10^{-14}$. Plot the errors of D_+ and D_0, compared with the value from (b), against δ on a log-log scale.

(d) Show that the error of D_+ is roughly $C_1\delta + \epsilon_{\text{mach}}/\delta$ and that of D_0 is roughly $C_2\delta^2 + \epsilon_{\text{mach}}/\delta$, where ϵ_{mach} is the machine precision. Find the best δ in each case (of order $\epsilon_{\text{mach}}^{1/2}$ and $\epsilon_{\text{mach}}^{1/3}$) and compare with your plot. Using these results and an operation count, explain why the gradient is computed by the chain rule and not by finite differences.

Problem 3: Training (25 points)

(a) Consider the least squares problem of Problem 1(c), with $L(\mathbf{u}) = \frac{1}{2m}\|\Phi\mathbf{u}-\mathbf{y}\|_2^2$ and minimizer $\mathbf{u}^*$. The gradient method with fixed step α is $\mathbf{u}^{k+1} = \mathbf{u}^k - \alpha\nabla L(\mathbf{u}^k)$. Let $H = \frac{1}{m}\Phi^T\Phi$ have eigenvalues $0 < \lambda_{\min} \le \cdots \le \lambda_{\max}$. Show that

$$\mathbf{u}^{k+1} - \mathbf{u}^* = (I - \alpha H)(\mathbf{u}^k - \mathbf{u}^*),$$

that the method converges for every $\mathbf{u}^0$ if and only if $0 < \alpha < 2/\lambda_{\max}$, and that the best choice of α gives the rate $(\kappa-1)/(\kappa+1)$, where $\kappa = \lambda_{\max}/\lambda_{\min}$.

(b) For equally spaced β_k and $n = 16$, run the gradient method with $\alpha\lambda_{\max} = 0.5$, 1.9, and 2.1, for at most 10^5 iterations, and plot $\|\mathbf{u}^k - \mathbf{u}^*\|_2$ against k on a semilog scale; take $\mathbf{u}^*$ from `qr`. Compare the observed rate with (a), and estimate the number of iterations needed for $n = 64$.

(c) Train the tanh network of Problem 2 with $n = 20$, starting from $a = 0$, $c = 0$, and w_k, b_k drawn from the standard normal distribution, by the following three methods.

- The gradient method with a fixed step α.
- The gradient method with momentum: $\mathbf{v}^{k+1} = \mu\mathbf{v}^k - \alpha\nabla L(\theta^k)$, $\theta^{k+1} = \theta^k + \mathbf{v}^{k+1}$, with $\mu = 0.9$.
- The Adam method: with $\mathbf{g} = \nabla L(\theta^k)$ and $\mathbf{m}^0 = \mathbf{v}^0 = 0$,

$$\mathbf{m}^{k+1} = \beta_1\mathbf{m}^k + (1-\beta_1)\mathbf{g}, \qquad \mathbf{v}^{k+1} = \beta_2\mathbf{v}^k + (1-\beta_2)\mathbf{g}^2,$$
$$\theta^{k+1} = \theta^k - \alpha\,\frac{\mathbf{m}^{k+1}/(1-\beta_1^{k+1})}{\sqrt{\mathbf{v}^{k+1}/(1-\beta_2^{k+1})} + 10^{-8}},$$

with $\beta_1 = 0.9$ and $\beta_2 = 0.999$; squares, square roots, and quotients are taken componentwise.

Use 2×10^4 iterations for each method, and choose α from $\{10^{-1}, 10^{-2}, 10^{-3}\}$. Plot $L(\theta^k)$ against k on a semilog scale for the three methods.

(d) Give E_∞ for each trained network and plot the networks together with f. Compare with the result of Problem 1(c) for a network with the same number of parameters. Repeat the best method with 5 different random starting values. Is L a convex function of θ? Do different starting values give the same final value of L? For this problem, is it better to train all parameters or only a and c?

Bonus: The Levenberg–Marquardt Method (10 points)

Write $L(\theta) = \frac{1}{2m}\|\mathbf{r}(\theta)\|_2^2$, where $\mathbf{r} \in \mathbb{R}^m$ has components r_i, and let $J(\theta) \in \mathbb{R}^{m \times (3n+1)}$ be the Jacobian matrix of $\mathbf{r}$ (its entries follow from Problem 2(a)). The Levenberg–Marquardt method solves

$$(J^T J + \lambda I)\,\mathbf{p} = -J^T \mathbf{r}$$

and sets $\theta \leftarrow \theta + \mathbf{p}$ if this decreases L, after which λ is divided by 10; otherwise the step is rejected and λ is multiplied by 10. Train the network of Problem 3(c) with this method and compare it with your best method from Problem 3(c), both in the number of iterations and in computing time. Why is the method fast here, and why is it seldom used for networks with millions of parameters?